

Ingenieria de Prompts: Guia Completa

Tecnicas, estrategias y buenas practicas para comunicarse con LLMs

1. Que es la Ingenieria de Prompts

La ingenieria de prompts es la disciplina que estudia como formular instrucciones efectivas para los modelos de lenguaje grande (LLMs). Un prompt bien disenado puede marcar la diferencia entre una respuesta vaga y una respuesta precisa, estructurada y util. A diferencia del fine-tuning, la ingenieria de prompts no requiere modificar los pesos del modelo: opera en tiempo de inferencia, lo que la hace accesible y rapida de iterar.

Los LLMs como GPT-4, Claude, Gemini o LLaMA son extremadamente sensibles a como se redactan las instrucciones. Pequeenos cambios en la formulacion pueden producir respuestas radicalmente diferentes. Por ello, la ingenieria de prompts es una habilidad critica para cualquier desarrollador o analista que trabaje con IA generativa.

2. Componentes de un Prompt

Un prompt completo suele contener cuatro elementos fundamentales:

- Instruccion: la tarea que debe realizar el modelo (resumir, clasificar, traducir, generar).
- Contexto: informacion adicional que ayuda al modelo a entender la situacion.
- Datos de entrada: el texto, codigo o datos sobre los que debe operar el modelo.
- Formato de salida: indicaciones sobre como debe estructurarse la respuesta (JSON, lista, tabla, etc.).

No todos los prompts necesitan los cuatro elementos. En tareas simples basta con la instruccion. En tareas complejas, cuanto mas contexto se proporcione, mejores resultados se obtienen.

3. Tecnicas Fundamentales

3.1 Zero-Shot Prompting

Consiste en pedir al modelo que realice una tarea sin proporcionar ejemplos previos. Es la tecnica mas sencilla y funciona bien en tareas que el modelo ya conoce del preentrenamiento. Ejemplo: 'Clasifica el siguiente texto como positivo, negativo o neutro: [texto]'. Los modelos modernos como GPT-4 o Claude 3.5 tienen excelentes capacidades zero-shot gracias a su entrenamiento a escala.

3.2 Few-Shot Prompting

Incluir dos o mas ejemplos de entrada-salida antes de la consulta real mejora drasticamente la precision del modelo en tareas con formato especifico. Los ejemplos actuan como un 'molde' que el modelo imita. Es especialmente util para tareas de clasificacion con etiquetas personalizadas, extraccion de entidades con formato especifico, o generacion de texto con un estilo particular.

3.3 Chain of Thought (CoT)

Añadir la frase 'Piensa paso a paso' o incluir ejemplos de razonamiento intermedio activa el pensamiento encadenado en el modelo. Esta tecnica mejora el rendimiento en problemas de logica, matematicas y razonamiento complejo. El modelo expone su proceso de razonamiento antes de dar la respuesta final, lo que reduce errores y hace la respuesta mas verificable.

3.4 Role Prompting (System Prompt)

Asignar un rol especifico al modelo modifica su comportamiento, tono y nivel de detalle. Ejemplos de roles efectivos: 'Eres un experto en ciberseguridad con 20 anhos de experiencia', 'Eres un profesor de matematicas que explica conceptos a alumnos de 12 anhos', 'Eres un asistente juridico especializado en derecho mercantil espanol'. El rol debe ser concreto y relevante para la tarea.

3.5 ReAct (Reason + Act)

ReAct es un paradigma que combina razonamiento con la capacidad de actuar (llamar herramientas externas). El modelo alterna entre pensar (Thought), actuar (Action) y observar resultados (Observation). Es la base de los agentes modernos con herramientas como busqueda web, calculadoras, APIs o bases de datos.

4. Buenas Practicas

- Ser especifico: instrucciones vagas producen respuestas vagas. Cuanto mas precisa la instruccion, mejor el resultado.
- Definir el formato de salida: si se necesita JSON, lista numerada o tabla, pedirlo explicitamente.
- Usar delimitadores: usar comillas triples (``), corchetes o XML para separar el contenido de las instrucciones.
- Iterar rapidamente: el prompting es un proceso empirico. Probar variaciones y medir resultados.
- Controlar la longitud: indicar 'en maximo 3 parrafos' o 'en una sola oracion' cuando sea relevante.
- Evitar negaciones dobles: en lugar de 'no hagas lo que no debes', usar 'haz X'.
- Temperatura segun la tarea: temperatura 0 para tareas deterministas (clasificacion, extraccion); mayor para creativas.

- Usar ejemplos negativos: mostrar lo que NO se quiere puede ser tan útil como los ejemplos positivos.

5. Errores Comunes

Los errores mas frecuentes en ingenieria de prompts son: instrucciones ambiguas que el modelo interpreta de formas inesperadas; pedir demasiadas cosas en un solo prompt sin estructurarlas; no especificar el publico objetivo (el nivel de tecnicidad cambia radicalmente la respuesta); asumir que el modelo conoce el contexto de la conversacion previa sin proporcionarselo; y no iterar sobre el prompt cuando los resultados no son satisfactorios.

6. Prompt Injection y Seguridad

El prompt injection ocurre cuando un input malicioso intenta sobrecribir las instrucciones del sistema. Es un riesgo critico en aplicaciones de produccion. Mitigaciones incluyen: separar claramente instrucciones del sistema de inputs del usuario, usar delimitadores robustos, validar y sanitizar inputs, y aplicar guardrails (filtros de salida) que detecten respuestas fuera del dominio esperado.

7. Prompting Avanzado: Meta-Prompts

Un meta-prompt es un prompt que genera prompts. Util para crear sistemas donde el modelo debe adaptar su propio comportamiento segun el contexto. Tambien existe el auto-prompting, donde el modelo itera y mejora su propio prompt hasta alcanzar un criterio de calidad. Estas tecnicas son la base de sistemas como DSPy o los optimizadores de prompts automaticos.